

避免 AI 循环思考与 Token 浪费的完整策略

一、已实施：loopDetection 机制

配置位置: `tools.loopDetection`

作用: 自动检测并干预工具调用循环

二、其他有效方法

2.1 思维链长度控制

方法: 通过 `thinking` 参数控制思维深度

```
// 关闭思维链（最节省）
"thinking": "off"

// 中等思维深度
"thinking": "medium"

// 仅在复杂任务启用高思维
"thinking": "high"
```

效果:

- `thinking=off`: 减少内部推理 token 消耗
- 适合简单任务、已知解决方案的场景

2.2 任务拆分与 Subagent 委托

方法: 将复杂任务拆分为小任务，委托给独立的 subagent

```
// 将大任务拆分
sessions_spawn({
  task: "只完成第一步：创建文件骨架",
  mode: "run", // 一次性任务
  timeoutSeconds: 120 // 设置超时
})
```

优势:

- ☒ 每个 subagent 有独立的上下文，不会累积历史
- ☒ 设置 `timeoutSeconds` 强制终止
- ☒ 任务边界清晰，避免在一个任务中循环
- ☒ 失败后可以重新 `spawn`，不会污染主会话

2.3 操作前状态检查（幂等性）

方法: 每次操作前检查是否已完成

```
# 创建文件前检查
if [ -f "target.java" ]; then
    echo "☑ 文件已存在, 跳过创建"
    exit 0
fi

# 修改前检查内容是否正确
if grep -q "expected_pattern" target.java; then
    echo "☑ 内容已正确, 跳过修改"
    exit 0
fi
```

效果: 避免重复创建/修改同一文件

2.4 设置超时限制

方法: 为所有耗时操作设置 timeout

```
// exec 超时
exec({
    command: "mvn compile",
    timeout: 300 // 5分钟超时
})

// subagent 超时
sessions_spawn({
    task: "...",
    timeoutSeconds: 120 // 2分钟超时
})
```

效果: 强制终止卡住的操作

2.5 明确终止条件

方法: 在任务开始时定义明确的完成标准

任务: 实现 X 功能
完成条件:

1. 文件 A.java 存在且包含方法 foo()
2. 测试通过: mvn test -Dtest=XTest
3. 代码编译无错误

效果: 有明确的"完成"定义, 避免无限迭代

2.6 人工干预点 (关键决策确认)

方法: 在关键决策点请求用户确认

```
"检测到以下情况需要决策:
1. 文件已存在, 是否覆盖?
2. 发现循环模式, 是否继续?
请回复 'y' 继续, 'n' 停止"
```

效果: 在可能陷入循环前让用户决定下一步

2.7 输出 Token 限制

方法: 设置 maxTokens 限制单次输出长度

```
{
  "model": {
    "maxTokens": 4096 // 限制单次输出不超过4K token
  }
}
```

效果: 强制精简输出, 避免冗长回复

2.8 定期进度检查 (Checkpoint)

方法: 在长任务中定期汇报进度并检查是否应该继续

```
每完成5个步骤:
- 汇报当前进度
- 检查是否偏离目标
- 询问是否继续
```

效果: 及时发现偏离, 避免无效工作累积

2.9 操作日志审查

方法: 记录所有操作, 定期审查发现重复模式

```
# 记录操作日志
echo "$(date) | $OPERATION | $TARGET | $STATUS" >> /tmp/agent-ops.log

# 检查最近是否有重复
tail -20 /tmp/agent-ops.log | grep "$OPERATION"
```

效果: 自我发现重复操作，主动调整

2.10 工具调用精简策略

方法: 减少不必要的工具调用

场景	优化方法
查询文件内容	先用 read，避免多次 read 同一文件
检查进程状态	用 process(action=poll, timeout=5000)，避免循环 poll
构建编译	一次 exec 完成多步，避免分多次调用
测试验证	合并测试命令，一次执行

2.11 上下文压缩 (Compaction)

方法: 启用自动上下文压缩

```
{
  "compaction": {
    "mode": "safeguard" // 自动压缩上下文
  }
}
```

效果: 防止历史累积导致 token 爆炸

2.12 模型选择策略

方法: 根据任务复杂度选择合适模型

任务类型	推荐模型	Token消耗
简单查询、文件操作	qwen3.6-plus	低
代码生成、分析	DeepSeek-pro	中
复杂推理、架构设计	doubao-seed-2.0-pro	高

效果: 避免"大模型做小任务"浪费

三、综合防护策略 (推荐组合)

层级1：系统级防护 (已配置)

- ☑ tools.loopDetection - 自动检测循环
- ☑ compaction.mode: safeguard - 自动压缩上下文

层级2：任务级防护（开发者实践）

- ☒ 任务拆分 + subagent 委托
- ☒ 设置 timeoutSeconds
- ☒ 明确终止条件

层级3：操作级防护（即时检查）

- ☒ 操作前状态检查（幂等性）
- ☒ 关键决策人工确认
- ☒ 定期进度汇报

层级4：模型级防护（参数优化）

- ☒ thinking=off（简单任务）
- ☒ maxTokens 限制
- ☒ 模型选择策略

四、最佳实践示例

示例1：复杂开发任务

1. 任务拆分（5个子任务）
2. 每个子任务用 sessions_spawn 委托
3. 设置 timeoutSeconds=120
4. 子任务完成后检查结果
5. 失败则重新 spawn，不污染主会话

示例2：文件创建任务

1. 先检查文件是否存在 [read]
2. 存在则检查内容是否正确 [grep]
3. 正确则跳过，输出"已完成"
4. 不正确则用 edit 精确修改
5. 修改后验证 [diff]

示例3：测试验证任务

1. 一次 exec 执行所有测试
2. 设置 timeout=300
3. 失败则分析日志 [process log]
4. 不循环重试，而是修复问题后重新测试

五、防护效果对比

方法	循环检测	Token节省	实施难度
loopDetection	★★★★★	★★★	低（配置即可）
任务拆分+subagent	★★★★★	★★★★★	中
状态检查（幂等性）	★★★★	★★★★	低
超时限制	★★★★	★★★	低
thinking=off	-	★★★★★	低
明确终止条件	★★★★	★★★	中
人工干预点	★★★★★	★★★	中
maxTokens限制	-	★★★★	低

六、总结

核心原则:

- 1. **预防 > 检测** - 操作前检查，避免重复
- 2. **拆分 > 整体** - 小任务更容易控制
- 3. **明确 > 模糊** - 有清晰的完成标准
- 4. **限制 > 无限** - 设置超时和token上限
- 5. **人工 > 自动** - 关键决策让用户参与

推荐组合:

系统防护 (loopDetection + compaction)
+ 任务拆分 (subagent + timeout)
+ 状态检查 (幂等性)
+ thinking控制
= 最有效的循环防护体系

一、架构与流程层面

1. 有状态记忆与去重

语义去重缓存：存储最近 N 轮用户输入 + Agent 输出的 embedding 或哈希值，当新步骤与历史步骤语义相似度超过阈值（如 0.95），判定为循环，直接中断或强制输出摘要。

动作哈希链：将 Agent 的每次动作（工具调用、API 参数、生成内容）计算为确定性哈希，连续相同哈希出现 M 次（如 3 次）即终止。

状态机约束：为 Agent 设计有限状态机（FSM），禁止从某个状态跳回前向状态（例如“查询结果状态”不允许回到“解析查询状态”）。

2. 外部看门狗进程

独立监控协程：在一轮对话中，启动一个轻量级后台任务，实时统计：

连续相同动作的次数

无输出变化的轮数

累计 Token 消耗

当超过阈值时强制 kill Agent 进程并返回兜底回答。

时间箱 Timebox：限制单次 Agent 对话的最大持续时间（如 30 秒），超时自动切断。

3. 预算硬上限

Token 预算：在调用 Agent 前分配 Token 预算（例如 2000），每个推理步骤累加输入+输出 Token，一旦超限立即停止并回退。

步数上限：硬限制最大推理轮数（如 10 轮），超出后输出“无法在指定步数内解决”。

二、提示工程与模型约束

1. 带步数引用的 Prompt 设计

在系统提示中要求 Agent：

每一步开始时输出 [Step X/MAX]，X 自动累加。

若某步输出与上一步完全相同，必须解释原因，否则强制终止。

在输出结尾添加 [CONTINUE] 或 [STOP] 标记，解析层据此决定是否继续。

2. 强制结尾模板

在提示词中设定结束条件白名单：

text

你只能在以下情况下停止：

- 已经成功完成用户任务
 - 已经尝试了至少3种不同方法且均失败
 - 用户明确要求停止
 - 达到了步数限制
- 不得因为“系统要求继续”而无限循环。

3. 自我反思链（Chain-of-Thought with Reflection）

要求 Agent 每执行 3 步后，插入一条反思：

“当前进度回顾：我完成了吗？之前的方法是否有重复？如果存在重复，我应当尝试新的策略或结束任务。”

这种元认知能显著降低陷入局部最优的概率。

三、工具调用层控制

1. 工具调用的频率限制

对同一工具、同一参数，连续调用次数上限（例如 search(query) 最多重复 2 次）。

引入退避延迟：每次重复调用后强制等待时间指数增长（100ms → 500ms → 2s），增加循环成本，促使其放弃。

2. 结果变化检测

在调用工具（如代码执行、API 请求）后，比较输出结果与前一次是否完全相同。若连续 2 次无变化，则：

不再将该结果返回给 Agent

直接回复“相同尝试再次执行，已阻止，请尝试其他方法”

3. 幂等性封装

对可能产生循环的操作（如发送消息、写数据库）设计 idempotency key，允许 Agent “重复调用”但实际只执行一次，避免副作用循环。

四、运行时主动干预

1. 多样性采样

当检测到生成内容重复时，强制修改解码参数：提高 temperature（如从 0.2 升到 1.0）或启用 top_p/top_k，迫使模型探索不同路径。

或送入一个“扰动提示”：追加“请用完全不同的思路解决”到当前对话。

2. 用户触发式熔断

在 Agent 每次输出后，允许用户输入特殊指令（如 /stop）立即终止循环，并退还剩余 Token 配额。

提供可视化“步数仪表盘”，让用户主动判断并停止循环。

3. 随机退避重试+自动放弃

若 Agent 尝试某动作失败，不是立即重试，而是随机等待 1~5 秒，重试最多 2 次。仍失败则标记该动作为“不可行”，从动作空间临时移除。

五、评估与离线防护

1. 离线仿真测试

在部署前，用历史易循环的 case 回放 Agent，检测：

最大重复步数

是否存在无进展轮转

Token 浪费比例

针对失败 case 调整 prompt 或阈值。

2. 在线日志分析与告警

记录每次对话的：

步数

同动作重复率

最终退出原因（正常 / 阈值熔断 / 预算耗尽）

设置异常告警：某用户或某场景在 1 小时内出现 3 次循环，自动触发限流或要求人工审查。

总结：最佳实践组合推荐

优先级 措施 成本 实现难度

1 Token 预算 + 步数硬上限 极低 易于实现

2 语义去重缓存（动作哈希） 低 中等

3 提示词强制反射与结束条件 无 简单

4 工具调用频率/结果变化检测 低 中等

5 外部看门狗进程 中 较高

推荐立即实施：步数限制（如 10 步）、Token 预算、相同输出连续出现 2 次后自动终止。这三者组合能挡住绝大多数循环浪费，且工程开销很小。

如果需要进一步讨论某个具体方案的实现细节（例如用 LangChain 如何加步数拦截），我可以给出示例代码。